

35.13. Choisir la taille du titre : `texteTitre(..., taille)`

Un 4^e argument, la **taille** : 2, 3 ou 4 cases de côté par lettre. Le même mot **TITRE**, dans les trois tailles, l'un sous l'autre.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : un quatrième argument, la taille. `texteTitre(colonne, ligne, "MOT", taille)` : la taille est le **nombre de cases de côté** d'une lettre. Sans lui, c'est 3, comme jusqu'ici.

Les trois tailles : **2** → une lettre de 16 × 16 pixels (2 colonnes, 2 lignes), **10 lettres** par ligne ; **3** → 24 × 24 pixels, **6 lettres** ; **4** → 32 × 32 pixels, **5 lettres**. Le style ne change pas : rondes, un contour noir, une ombre, et elles sautillent. En taille 2, le contour est plus fin et les coins ne sont pas arrondis : la place manque.

Le calcul de la place : une lettre prend « taille » colonnes. **TITRE** (5 lettres) prend donc 5 × 2 = **10** colonnes en taille 2, 5 × 3 = **15** en taille 3, 5 × 4 = **20** en taille 4 — tout l'écran. Et « taille » lignes : en lignes 0 à 1, puis 3 à 5, puis 8 à 11.

La taille s'écrit en clair — 2, 3 ou 4 —, **jamais avec une variable** : les lettres sont **dessinées par le compilateur**, avant que le jeu ne tourne, donc il doit connaître leur taille en lisant le programme. Avec une variable, ou une taille 5, le compilateur refuse et dit pourquoi.

Ce qu'il coûte : chaque taille a ses propres dessins. Le T en taille 2 et le T en taille 4 ne sont pas les mêmes tuiles : un titre dans les trois tailles coûte trois fois. Dans un vrai jeu, on en choisit une.

À toi : écris le nom de ton jeu dans la taille qui lui va. Dix lettres ? Taille 2. Un nom court qui doit frapper ? Taille 4.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texteTitre>` → [35.11. La fonction `texteTitre\(\)` — un mot en GROSSES lettres de titre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2355 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         721 o ← démarrage - ta fonction
VBlank         8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
CopierTuiles   31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         1488 o ← CopierTuiles - 0 lettre sur 43, et 93 tuiles dessinées
Lettres gravées : aucune
```

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()      x 119   714 o sur place, dans main()
image()      x   1    3 o sur place, dans main() + AttendreImage (28 o)
texteTitre() x   3    0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Le même mot, TITRE, dans les trois tailles : le 4e argument de texteTitre().
```

```
#include <texteTitre> // texteTitre(colonne, ligne, "MOT", taille)
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // taille 2 : chaque lettre fait 2 x 2 cases → colonnes 0 à 9, lignes 0 à 1
    texteTitre(0, 0, "TITRE", 2);

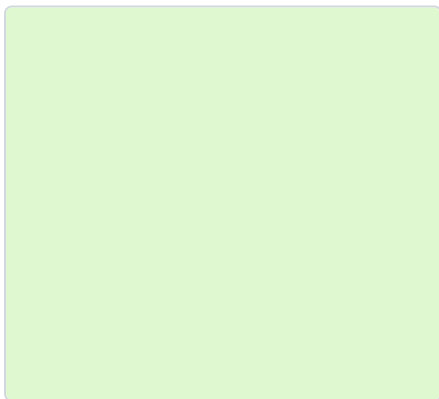
    // sans 4e argument : taille 3, 3 x 3 cases → colonnes 0 à 14, lignes 3 à 5
    texteTitre(0, 3, "TITRE");

    // taille 4 : 4 x 4 cases → colonnes 0 à 19 (tout l'écran), lignes 8 à 11
    texteTitre(0, 8, "TITRE", 4);

    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois fois TITRE, de plus en plus grand : petit en haut (la moitié de l'écran), moyen au milieu, et en bas, grand, sur toute la largeur.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ taille 2 : TITRE en colonnes 0 à 9, et rien après
 - ✓ taille 3 : TITRE en colonnes 0 à 14, et rien après
 - ✓ taille 4 : TITRE sur toute la largeur, jusqu'à la colonne 19
 - ✓ chaque taille a ses propres dessins (45, 64, 92)
-

Ensuite : **35.14. La fonction `texteManga()` — un titre entre manga et dessin animé** — chapitre 6 : Dessiner. · [gameboy3](#) — écrire une cartouche Game Boy en C++